

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	M.Madhu Rushwika	Reg. No.:	192371066
Section / Batch:	cse	Date:	16-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Financial Analytics System

A financial system uses an algorithm with recurrence $T(n) = 2T(n/2) + n \log n$. Apply the Master Theorem and determine the complexity. Compare $f(n)$ with

$n^{\log_b a}$ and justify the correct case. Analyze performance for large datasets. Interpret efficiency clearly.

Q2. Banking Transaction Processing System A banking system uses a divide-and-conquer algorithm $T(n) = 4T(n/2) + n^2$. Apply the Master Theorem to determine the time complexity. Identify parameters and justify the case selection. Analyze its performance for large transaction volumes. Interpret efficiency using asymptotic notation.

Code of Conduct

I M.Madhu Rushwika (192371066) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Madhu Rushwika

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Assignment: Master Theorem Analysis of Divide-and-Conquer Algorithms

Introduction

Divide-and-conquer is one of the most important algorithm design techniques in computer science. In this approach, a large problem is divided into smaller subproblems of the same type. Each subproblem is solved recursively, and the solutions are combined to obtain the final result. Many efficient algorithms, such as Merge Sort, Quick Sort, Binary Search, and Strassen's Matrix Multiplication, use this technique.

To analyze the running time of divide-and-conquer algorithms, recurrence relations are used. Solving these recurrences manually is often difficult. The **Master Theorem** provides a direct mathematical method to determine the time complexity of recursive algorithms.

The general recurrence equation is:

$$T(n) = aT(n/b) + f(n)$$

where:

- **a** = Number of recursive subproblems
- **b** = Factor by which the problem size is reduced
- **f(n)** = Cost of dividing and combining the subproblems

The Master Theorem compares **f(n)** with $n \log_b a$ and determines the asymptotic complexity.

Master Theorem

The Master Theorem consists of three cases.

Case 1

If

$$f(n) = O(n^{\log_b a - \epsilon})$$

for some constant $\epsilon > 0$,

then

$$T(n) = \Theta(n^{\log_b a})$$

This means the recursive work dominates the total running time.

Case 2

If

$$f(n) = \Theta(n \log b \log k n) \quad f(n) = \Theta(n \log b \log k n) \quad f(n) = \Theta(n \log b \log k n)$$

then

$$T(n) = \Theta(n \log b \log k + 1 n) \quad T(n) = \Theta(n \log b \log k + 1 n) \quad T(n) = \Theta(n \log b \log k + 1 n)$$

Here, both recursive work and non-recursive work contribute equally.

Case 3

If

$$f(n) = \Omega(n \log b a + \epsilon) \quad f(n) = \Omega(n \log b a + \epsilon) \quad f(n) = \Omega(n \log b a + \epsilon)$$

and the regularity condition holds,

then

$$T(n) = \Theta(f(n)) \quad T(n) = \Theta(f(n)) \quad T(n) = \Theta(f(n))$$

In this case, the non-recursive work dominates.

Importance of Master Theorem

The Master Theorem is important because:

- It quickly solves recurrence relations.
- It avoids lengthy substitution methods.
- It helps compare recursive algorithms.
- It predicts algorithm performance.
- It is widely used in algorithm design and analysis.

Applications include:

- Financial analytics
- Banking systems

- Cloud computing
- Artificial Intelligence
- Machine Learning
- Image Processing
- Big Data Analytics
- Database Systems

Q5. Financial Analytics System

Problem Statement

A financial system uses the recurrence

$$T(n) = 2T(n/2) + n \log n$$

Apply the Master Theorem to determine the time complexity. Compare $f(n)$ with $n \log^a n$ and justify the correct case.

Step 1: Identify Parameters

Given,

- $a = 2$
- $b = 2$
- $f(n) = n \log n$

Step 2: Compute $n \log^a n$

$$n \log^2 n = n (\log_2 n)^2$$

Therefore,

$$n \log^2 n = n (\log_2 n)^2$$

Step 3: Compare Functions

Given

$$f(n) = n \log n \quad f(n) = n \log n \quad f(n) = n \log n$$

and

$$n \log b a = n n^{\log_{ba}} = n n \log b a = n$$

Since

$$n \log n = \Theta(n \log^1 n) \quad n \log n = \Theta(n \log^1 n) \quad n \log n = \Theta(n \log^1 n)$$

Therefore,

$$f(n) = \Theta(n \log b a \log^1 n) \quad f(n) = \Theta(n^{\log_{ba}} \log^1 n) \quad f(n) = \Theta(n \log b a \log^1 n)$$

Hence,

Master Theorem Case 2 applies.

Step 4: Apply Master Theorem

According to Case 2,

$$T(n) = \Theta(n \log b a \log^{k+1} n) \quad T(n) = \Theta(n^{\log_{ba}} \log^{k+1} n) \quad T(n) = \Theta(n \log b a \log^{k+1} n)$$

where

$$k = 1 \quad k = 1 \quad k = 1$$

Therefore,

$$T(n) = \Theta(n \log^2 n) \quad \boxed{T(n) = \Theta(n \log^2 n)} \quad T(n) = \Theta(n \log^2 n)$$

Recursion Tree Analysis

Root Level

Cost:

$$n \log n \quad n \log n \quad n \log n$$

First Level

There are two subproblems.

Each costs

$$n^2 \log n^2 \frac{n^2 \log n^2}{n^2} = 2n \log 2n$$

Total cost

$$2 \times n^2 (\log n - 1) = n (\log n - 1)^2 \times \frac{n^2 (\log n - 1)}{n (\log n - 1)} = n (\log n - 1)^2 \times 2n (\log n - 1) = n (\log n - 1)$$

Second Level

Four subproblems

Total cost

$$n (\log n - 2) n (\log n - 2) n (\log n - 2)$$

Every level decreases the logarithmic term by one.

The recursion continues for

$$\log n \log n \log n$$

levels.

Adding all costs,

$$n[(\log n) + (\log n - 1) + (\log n - 2) + \dots] n[(\log n) + (\log n - 1) + (\log n - 2) + \dots] n[(\log n) + (\log n - 1) + (\log n - 2) + \dots]$$

which simplifies to

$$\Theta(n \log^2 n) \boxed{\Theta(n \log^2 n)} \Theta(n \log^2 n)$$

Performance Analysis

Small Datasets

- Very fast execution.
- Low recursion overhead.
- Efficient memory utilization.

Large Datasets

- Logarithmic growth keeps execution time manageable.
- Suitable for processing millions of financial records.
- Used in portfolio optimization, fraud detection, investment analysis, and financial forecasting.

Advantages

- High scalability
- Efficient divide-and-conquer approach
- Suitable for parallel execution
- Better than quadratic algorithms
- Handles massive datasets effectively

Limitations

- Slightly slower than $O(n \log n)$
- Extra logarithmic factor increases execution time.

Conclusion

The recurrence satisfies **Master Theorem Case 2**.

Hence,

$$T(n) = \Theta(n \log^2 n) \quad \boxed{T(n) = \Theta(n \log^2 n)}$$

The algorithm is efficient and scalable for modern financial analytics applications.

Q6. Banking Transaction Processing System

Problem Statement

A banking transaction system uses

$$T(n) = 4T(n/2) + n^2$$

Determine the time complexity using the Master Theorem.

Step 1: Parameters

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Step 2: Compute

$$n \log_b a = n \log_2 4 = n^2$$

Step 3: Compare

Since

$$f(n) = n^2$$

and

$$n \log_b a = n^2$$

Therefore,

$$f(n) = \Theta(n \log_b a)$$

Hence,

Master Theorem Case 2 applies.

Step 4: Apply Master Theorem

Using Case 2,

$$T(n) = \Theta(n \log_b a \log n) = \Theta(n^2 \log n)$$

Therefore,

$$T(n) = \Theta(n^2 \log n) \quad \boxed{T(n) = \Theta(n^2 \log n)}$$

Recursion Tree Analysis

Level 0

Cost

$$n^2$$

Level 1

There are four subproblems.

Each costs

$$\left(\frac{n}{2}\right)^2 = \frac{n^2}{4} \quad (2n)^2 = 4n^2$$

Total

$$4 \times \frac{n^2}{4} = n^2$$

Level 2

There are sixteen subproblems.

Each costs

$$\left(\frac{n}{4}\right)^2 = \frac{n^2}{16} \quad (4n)^2 = 16n^2$$

Total

$$16 \times \frac{n^2}{16} = n^2$$

Every level contributes

$$n^2$$

There are

$\log n$ levels.

Therefore,

$$\Theta(n^2 \log n) \boxed{\Theta(n^2 \log n)} \Theta(n^2 \log n)$$

Performance Analysis

Small Banking Databases

- Fast processing
- Low response time
- Efficient recursive computation

Large Banking Databases

- Runtime increases because of the quadratic factor.
- Suitable for medium-sized transaction systems.
- Less efficient than $O(n \log n)$ algorithms for extremely large banking databases.

Applications include:

- Transaction verification
- Loan processing
- Fraud detection
- Banking audit systems
- Batch transaction processing

Advantages

- Simple recursive structure
- Predictable running time
- Easy mathematical analysis
- Suitable for structured transaction processing

Limitations

- Quadratic term dominates execution time.
- Poor scalability compared with linearithmic algorithms.

Comparison of Q5 and Q6

Feature	Q5	Q6
Recurrence	$2T(n/2) + n \log n$ $2T(n/2) + n \log n$	$4T(n/2) + n^2$ $4T(n/2) + n^2$
Master Theorem Case	Case 2	Case 2
Time Complexity	$\Theta(n \log^2 n)$ $\Theta(n \log^2 n)$	$\Theta(n^2 \log n)$ $\Theta(n^2 \log n)$
Scalability	Excellent	Moderate
Suitable For	Very large datasets	Medium to large datasets

Conclusion

The Master Theorem is a powerful technique for solving divide-and-conquer recurrences. In both problems, **Case 2** applies because the non-recursive work matches $n \log^b a$ up to logarithmic factors. Among the two algorithms, the **Financial Analytics System** with $\Theta(n \log^2 n)$ is more scalable and efficient for very large datasets, while the **Banking Transaction Processing System** with $\Theta(n^2 \log n)$ is suitable for moderate-scale transaction processing but becomes more computationally expensive as the input size grows. This analysis helps in selecting efficient algorithms for real-world financial and banking applications.

